SynapseOS: Designing a Conversational Session Layer for the Linux Desktop

by

Alexandra Sulit
Willard Soriano
Allyson Vivar

A Thesis Proposal Submitted to the Department of Computer Science
in Partial Fulfillment of the Requirements for the Degree
Bachelor of Science in Computer Science

Mapúa University – Makati
July 2026

APPROVAL PAGE

[To be inserted upon approval.]

ACKNOWLEDGEMENT

[To be inserted prior to final submission.]

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

ABSTRACT

This study designs, implements, and evaluates SynapseOS, a conversational session layer for the Linux desktop that replaces the conventional graphical session — the desktop shell, session manager, and application launcher — with a natural-language interface powered by a locally-hosted small language model (Qwen2.5-Coder-3B-Instruct). SynapseOS translates natural-language commands into shell operations executed through the standard Linux command-line toolchain, incorporating a reversibility-based confirmation gate for irreversible operations and an undo mechanism for recoverable ones. The system is evaluated through a controlled, within-subjects user study comparing task completion time, error rate, and user satisfaction between SynapseOS and each participant's own primary operating system across novice and power user populations (n = 20). This evaluation methodology addresses a recognized gap in the literature, where conversational computing interfaces are rarely compared against conventional graphical workflows on the same tasks and population.

Keywords: conversational interface, session layer, natural language processing, Linux desktop, user study

Chapter 1

INTRODUCTION

The dominant paradigm for personal computing has remained largely unchanged since the early 1980s, when the graphical desktop — icons, menus, and pointer-driven interaction — was established through the Xerox Star and Apple Lisa lineage. Despite its longevity, a parallel research tradition has long explored whether users should instead be able to describe what they want in natural language and have the computer carry it out. The earliest serious system in this tradition, the Berkeley UNIX Consultant, was already operational by the late 1980s [1]. Decades later, the rise of large language models (LLMs) and multimodal foundation models[1] has reopened this thread with substantially greater technical capability.

The field that has emerged sits at the intersection of human-computer interaction, operating systems, natural language processing, and computer vision. On the shell side, recent work has demonstrated that LLMs can translate natural language into Bash commands with usable accuracy and can be embedded as a Unix primitive with appropriate safety guardrails [2, 3]. On the graphical side, so-called computer-using agents[2] now operate desktop and mobile applications by combining native APIs with vision-based grounding [16, 18]. On the web, language-guided agents complete tasks across realistic websites with measurable but limited reliability [8, 9]. A converging research thread approaches the same technical problem from an accessibility perspective: speech-driven GUI command mapping for users with motor and speech impairments has progressed from voice-to-text dictation toward voice-to-action agents, with recent work on speech-instructed GUI agents [22] and region-aware visual grounding [23] demonstrating that the underlying components are maturing rapidly. Yet despite this progress across both threads, the most rigorous evaluation environment in the field — the OSWorld benchmark — shows that frontier models complete only 12.24% of real computer tasks that humans complete at 72.36% [17]. The gap between demonstration and deployable replacement remains substantial.

This rapid progress has been uneven across platforms. Windows is heavily represented in the desktop agent literature [15, 16], Android in the mobile agent literature [10, 11, 12], and the browser in the web agent literature [7, 8, 9]. Linux desktop coverage, by contrast, is conspicuously absent: existing work either addresses Linux only at the shell level [3] or treats it as one of several evaluation platforms rather than a primary target [17]. No surveyed system implements a working conversational session layer[3] for the Linux desktop, despite Linux serving as the substrate for most developer and server-side workflows.

Gap / Opportunity

Three compounding gaps in the existing literature motivate the present research. First, no system in the literature has replaced the conventional desktop session layer with a conversational one — existing systems automate on top of the desktop shell rather than replacing it. The UFO² system, for instance, positions itself as a "Desktop AgentOS" but operates as an automation layer atop the existing Windows shell rather than substituting for it [16]. Second, Linux desktop coverage remains absent despite Linux being the substrate for most developer and server-side workflows; the NaSh project addresses Linux but only at the shell level [3], and OSWorld includes Ubuntu only as one of three evaluation platforms rather than as a primary deployment target [17]. Third, no existing work compares a conversational interface against the conventional graphical workflow on the same tasks with the same user population [17]; user studies in this space are rare, with only VoicePilot [5] conducting a substantive human-centered evaluation of LLM-based interaction.

Notably, the accessibility-focused thread of the field exhibits the inverse limitation. Speech-to-GUI mapping research targets specific user populations — users with motor impairments, speech disorders, or cognitive disabilities — with specialized techniques such as speech-instructed GUI agents and resilient visual grounding [22, 23], but operates at the level of individual applications or interaction techniques rather than at the level of the session layer itself. These approaches presuppose an underlying conversational system surface on which to run, yet no such surface exists. This complementarity sharpens the opportunity: a general-purpose conversational session layer would serve not only mainstream users but also provide the substrate that accessibility-specific configurations require.

These gaps present a clear opportunity, and a precedent for how to seize it. The history of the Unix shell demonstrates that a new interactive layer — bash succeeding the Bourne shell, zsh and fish succeeding both — can replace the software a user directly interacts with while leaving the kernel and the core system utilities entirely unchanged, producing a materially different experience for the user without touching anything beneath the session. The opportunity addressed by this study is to apply the same pattern in a new direction: SynapseOS replaces the conventional graphical session layer — the desktop shell, window manager, and application launcher — with a conversational one, leaving the kernel, the core system utilities, and the existing Linux application ecosystem untouched. It is designed to serve as the primary user-facing interface to a personal computing environment — spanning novice, elderly, non-technical, and technical user populations — with an evaluation methodology that engages the conventional desktop workflow as its baseline rather than as its substrate.

Problem Statement

This study addresses the absence of a conversational session layer for the Linux desktop — a conversational shell replacing the desktop's graphical session in the way a new interactive shell replaces the one before it, rather than a modification to the kernel or the core system utilities — that has been empirically evaluated against the conventional graphical workflow it would displace.

Specifically, the study seeks to answer the following questions:

1. How can a conversational session layer be designed to function as the primary user-facing interface to a Linux-based personal computing environment, spanning the full range of shell-expressible operations?
2. What natural-language-to-bash intent parsing and execution design — incorporating a reversibility-based confirmation gate and an undo mechanism — is most effective for grounding natural language intent into safe, recoverable system actions on the Linux command line?
3. How does SynapseOS compare to each participant's own primary operating system in terms of task completion time, error rate, and user satisfaction across novice and power user populations?

Objectives

The general objective of this study is to design, implement, and evaluate SynapseOS, a system that replaces the conventional graphical session layer — the desktop shell, session manager, and application launcher — with a conversational interface layer, while leaving the underlying Linux kernel, core system utilities, and application ecosystem unchanged.

Specifically, this study aims:

1. To design a conversational session layer for the Linux desktop that spans the full range of shell-expressible operations under a single natural-language surface.
2. To develop a natural-language-to-bash intent parsing and execution pipeline for SynapseOS, incorporating a reversibility-based confirmation gate and an undo mechanism, balancing intent parsing accuracy against execution safety.
3. To implement SynapseOS as a Linux session manager[4] that provides a persistent conversational surface, a structured undo log, and a confirmation policy for irreversible operations, addressing the safety and recoverability gaps identified in prior work [3, 16].
4. To evaluate SynapseOS through a controlled user study comparing task completion time, error rate, and self-reported satisfaction against each participant's own primary operating system, across novice and power user populations, providing the empirical comparison absent from the existing literature [17].

Significance of the Study

The findings of this study are expected to benefit the following groups:

1. **Novice and non-technical computer users.** The conventional graphical desktop imposes a steep learning curve rooted in its icon-and-menu paradigm. A conversational session layer that accepts natural language intent lowers the threshold for productive computer use, particularly for users with limited prior computing experience, including elderly users encountering desktop computing late or infrequently.
2. **Developers and power users on Linux.** A conversational session layer reduces the cognitive load of recalling and composing shell commands by exposing system operations through a unified natural-language interface.

3. **Users with motor accessibility needs.** Although this population is scoped out of the initial evaluation for ethical and recruitment reasons, SynapseOS is positioned to serve as the session-level substrate on which accessibility-specific configurations can operate. Prior work has established design guidelines for LLM-based speech interfaces for users with motor impairments [5] and has demonstrated specialized techniques — speech-instructed GUI agents and resilient visual grounding — at the application and interaction-technique level [22, 23]. These techniques presuppose a conversational system surface that does not yet exist at the session level; SynapseOS provides that surface, making accessibility-specific extensions a natural and well-grounded future direction rather than a speculative one.
4. **The research community.** The controlled comparative evaluation methodology this study introduces addresses a recognized gap in the literature, where conversational interfaces are rarely compared against conventional graphical workflows on the same tasks and population [17]. The evaluation design provides a replicable methodology for future work in this space.

1.1 Scope and Limitations of the Study

This study covers the design, implementation, and evaluation of SynapseOS as a conversational session layer for the Linux desktop. SynapseOS replaces the conventional graphical session — the desktop shell, session manager, and application launcher — with a conversational interface, analogous in scope to how a new interactive shell replaces the one before it: the kernel and the core system utilities beneath the session are unchanged. The Linux kernel, the core system utilities, and the existing ecosystem of Linux applications are not modified by this study; SynapseOS interacts with the system by translating natural language into shell commands executed through the standard command-line toolchain, rather than by rewriting applications or the utilities they depend on. The subject matter spans natural language processing, LLM-based reasoning, command-line automation, Linux system integration, and human-computer interaction.

The study examines cross-application desktop workflows on Linux, including shell operations, file management, and application-level tasks representable in the OSWorld benchmark task suite [17]. The population studied consists of novice computer users and power users recruited for a controlled user study. The study is conducted in a Linux desktop environment and is bounded by the duration of the thesis project.

The following limitations are acknowledged. First, SynapseOS commits to a locally-hosted, code-specialized small language model (Qwen2.5-Coder-3B-Instruct) rather than a hosted frontier model, prioritizing privacy and offline operation over the potentially higher capability of larger hosted alternatives[5]; this tradeoff may affect the ceiling on intent parsing accuracy achievable within the study. Second, users with motor accessibility needs — a highly relevant population — are excluded from the initial evaluation due to ethics-review and recruitment constraints; accessibility-specific configurations such as speech-instructed agent operation for atypical speech patterns [22] are likewise out of scope for the initial implementation but represent a natural extension of the SynapseOS substrate. Third, SynapseOS targets the Linux desktop specifically, and the findings may not generalize directly to Windows or macOS desktop environments. Fourth, the safety confirmation policy for

irreversible system operations is defined in principle but requires empirical calibration during implementation. Fifth, evaluation is bounded by the task coverage of the OSWorld benchmark and the custom task suite developed for this study; tasks outside this coverage are not evaluated.

Notes

1. Foundation models are large neural networks trained on broad, general-purpose data and adaptable to many tasks. Multimodal variants accept multiple input types — typically text and images — enabling the model to interpret visual interfaces and screenshots alongside natural language.
2. Computer-using agents are AI systems that interact with a computer's graphical environment — clicking, typing, reading the screen — to complete tasks on behalf of a user. Unlike shell-level automation, they operate through the same visual interface a human would use.
3. The session layer (or simply, the session) is the software the user directly interacts with immediately after login — the desktop shell, window manager, session manager, and application launcher on a graphical system, or the login shell on a text terminal. It is distinct from the userland (or user space): the core system libraries and command-line tools (e.g., glibc, coreutils, bash, apt) that run outside the kernel and that the session layer itself depends on and invokes. SynapseOS replaces the session layer; the underlying userland is unchanged.
4. A Linux session manager is the program that starts after user login and establishes the graphical environment — launching the window manager, desktop shell, and background services. SynapseOS replaces this component with a conversational interface, making it the first program a user encounters after login.
5. Open-weights models make their trained parameters publicly available for local download and use. Users run inference on their own hardware without routing queries to an external service provider, enabling offline operation and preventing user data from being transmitted to third parties.

Chapter 2

REVIEW OF RELATED LITERATURE

2.1 Introduction to the Review

This chapter reviews the body of research relevant to the design and evaluation of a conversational session layer for the Linux desktop. The review spans nearly four decades of work on natural-language interfaces to computing systems, from the symbolic-artificial-intelligence consultants developed for the Unix shell in the late 1980s [1] to the large language model (LLM) agents that now operate shells, graphical applications, browsers, and mobile devices. Its purpose is threefold: to establish the conceptual vocabulary and technical foundations on which the present study builds, to characterize the current state of the field and its recurring limitations, and to locate precisely the gap that the proposed system, SynapseOS, is designed to fill.

2.2 Theoretical Framework

The heterogeneous systems surveyed in this chapter — shell translators, web and mobile agents, desktop automation frameworks, and accessibility interfaces — differ widely in platform, action space, and evaluation method, yet they address a common problem: converting a user's expressed intent into effects on a computing system. To compare them on common ground, this review adopts a *four-layer interaction pipeline* — Input, Reasoning, Action, and Operating-System Integration — as its theoretical framework. The framework is adapted from the layered decomposition of large language model (LLM)-brained graphical user interface (GUI) agents proposed in the survey of Zhang and colleagues [18], generalized here from graphical agents specifically to language-mediated computing interfaces as a class. Figure 2.1 depicts the pipeline and situates the layers relative to one another.

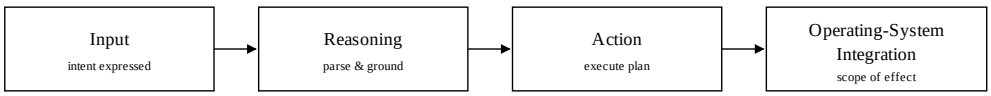

Figure 2.1. The four-layer interaction pipeline for language-mediated computing interfaces, adapted from the LLM-brained GUI-agent survey of Zhang and colleagues [18]. A user's intent enters at the Input layer, is parsed and grounded at the Reasoning layer, executed at the Action layer, and takes effect within a scope defined at the Operating-System Integration layer.

The *Input* layer concerns how a user expresses intent, whether by typed text, speech, or structured shortcuts. The *Reasoning* layer concerns how that intent is parsed into a plan and grounded into the affordances of the environment. The *Action* layer concerns how a grounded plan is executed — through command translation, through binding to native application programming interfaces (APIs), or through simulated keyboard and mouse operations. The *Operating-System Integration* layer concerns the scope of computing an approach can affect, from a single application to an entire operating system.

This framework is appropriate to the present study for three reasons. First, it is *modality-neutral*: because it abstracts over whether intent arrives as text, speech, or gesture and whether action is realized through shell commands, API calls, or simulated clicks, it accommodates the full span of surveyed systems without privileging any one interaction style — a necessary property for a review that spans command-line, graphical, and accessibility work. Second, it is *diagnostic*: locating each system's principal limitation at a specific layer — visual grounding as a Reasoning-and-Action failure, or narrow platform coverage as an Operating-System Integration limitation — turns an otherwise flat catalogue of systems into a structured comparison, and it is this layer-by-layer positioning that the comparative synthesis of §2.8 makes explicit. Third, it is *directly applicable* to the proposed system: the conceptual framework of SynapseOS presented in Chapter 3 instantiates this same pipeline — accepting natural-language intent at the Input layer, parsing it with a locally-hosted small language model at the Reasoning layer, translating it into shell commands executed through the standard toolchain at the Action layer, and spanning the full graphical session rather than a single application at the Operating-System Integration layer. The framework therefore serves both as the organizing lens for the literature and as the vocabulary in which the study's own contribution is later positioned.

Works were included in this review if they satisfy at least one of four criteria: they translate natural language into executable system actions; they operate graphical, web, or mobile interfaces through LLM-based reasoning; they address speech- or accessibility-driven interaction with computing systems; or they provide an evaluation environment for such systems. Works concerned solely with model architecture or training, without a computing-interaction target, and general-purpose conversational systems without a system-control capability, were excluded as outside the scope of this study. The chapter first sets out the theoretical framework that organizes the review (§2.2), then proceeds thematically — historical foundations (§2.3), the shell and command-line thread most directly related to the proposed design (§2.4), graphical and computer-using agents (§2.5), accessibility and speech-driven interaction (§2.6), and evaluation environments (§2.7) — before synthesizing the surveyed works into a comparative analysis and an explicit statement of the research gap (§2.8). It closes with a review of the methodological literature that informs the study's evaluation design (§2.9) and a summary that motivates the methodology detailed in Chapter 3 (§2.10).

2.3 Historical Foundations of Natural-Language Interfaces

The aspiration to let users describe what they want in ordinary language and have a computer carry it out is not new. The earliest serious system in this tradition, the Berkeley Unix Consultant of Wilensky and colleagues [1], was already operational by the late 1980s. The Unix Consultant was an intelligent natural-language interface that allowed naive users to learn about the Unix operating system through conversation in ordinary English, addressing both the natural-language-frontend problem and the consultation problem, and thereby distinguishing itself from systems intended only as command translators. It was built on the KODIAK knowledge representation and incorporated a user-modeling component that represented the user's knowledge state and a teaching component through which users could extend its knowledge base in natural language. The system demonstrated end-to-end natural-language consultation about Unix, successfully answering "how do I," definitional, and problem-solving questions. Critically, its authors identified knowledge acquisition as the principal bottleneck: extending the system to a new domain required substantial hand-engineering — a concern that directly foreshadows the contemporary problem of LLM-based agents requiring per-application grounding data.

The intervening decades saw natural-language interaction treated primarily as a supervised-learning problem. Representative of this pre-LLM era, the work of Li and colleagues on mapping natural-language instructions to mobile user-interface action sequences [19] trained dedicated phrase-extraction and grounding transformers on hand-annotated action sequences, achieving 70% partial accuracy on its benchmark but remaining bound to its purpose-built datasets and architectures. The decisive shift came with the introduction of in-context reasoning patterns for LLMs. ReAct, introduced by Yao and colleagues [20], collapsed the multi-stage supervised pipeline into a prompting strategy: the language model is prompted to alternate between free-text "Thought" steps that update its plan and "Action" steps that interact with an external environment, with observations fed back into subsequent reasoning. Implemented purely as a prompting strategy without fine-tuning, ReAct nonetheless outperformed imitation-learning and reinforcement-learning baselines that required thousands of training trajectories.

The historical significance of this transition is twofold, and it frames the remainder of this review. First, it explains why progress in the field has been so rapid since 2023: the methodological foundations became portable across modalities and platforms in a way that the earlier dataset-bound approaches were not. Nearly every system surveyed in the sections that follow inherits from ReAct the convention of interleaving reasoning traces with environment-affecting actions. Second, this shared inheritance underscores a corresponding fragility: these systems inherit the failure modes of LLM reasoning — hallucinated actions, plan inconsistency, and brittleness on long-horizon tasks — alongside their capabilities, a concern that the evaluation literature reviewed in §2.7 quantifies.

2.4 Natural-Language-to-Shell Translation

The thread of the literature most directly related to the proposed system concerns the translation of natural language into shell commands. Because SynapseOS grounds user intent into commands executed through the standard Linux command-line toolchain, the accuracy, safety, and context-awareness of natural-language-to-Bash translation are of central relevance to its design.

Westenfelder and colleagues [2] address this translation problem directly. Their work constructs a manually verified test set of 600 instruction–command pairs and a training set of 40,939 pairs, and introduces a functional-equivalence heuristic that reaches 95% confidence in determining whether two Bash commands produce the same effect — allowing automated evaluation to move beyond exact-string matching to a measure of whether a generated command actually accomplishes the intended task. Their fine-tuned models substantially outperform zero-shot baselines, demonstrating that fine-tuning specifically on command-translation data is effective. The authors identify two persistent limitations: ambiguity in user instructions, and the absence of system context — the contents of the current directory, the user's command history, and environment variables — at translation time. Integrating context-aware retrieval is identified as a direction, one that the persistent conversational session of SynapseOS is well positioned to supply.

Where translation accuracy is the concern of Westenfelder and colleagues, safety is the concern of Gyawali and colleagues, whose NaSh system [3] proposes guardrails for an LLM-powered natural-language shell for Linux. Published under the operating-systems classification, NaSh interposes a guardrail layer between the user's natural-language input and the LLM-generated command: the model proposes a candidate command, the guardrail layer evaluates it against safety, reversibility, and confirmation criteria, and the user receives a structured preview before execution. The authors note that LLM outputs may be unintended or unexplainable, requiring guardrails for user error recovery, and they leave as open work both the design of recovery and rollback mechanisms for irreversible system operations and the empirical user evaluation of the approach. These two open problems — reversibility-aware confirmation and empirical validation with users — are precisely the concerns that the confirmation gate, undo mechanism, and controlled user study of the present study are designed to address.

The feasibility of deploying such a system without reliance on a hosted frontier model is supported by the large-scale evaluation of Jacobs and colleagues [25], who assess a broad range of locally deployable open-source LLMs on Bash command generation. Their work provides an empirical anchor for the viability of running natural-language-to-Bash translation on local, open-weights models of modest size — the deployment posture adopted by SynapseOS for reasons of privacy and offline operation. Taken together, the shell-and-command-line thread establishes that natural-language-to-Bash translation is tractable, that safety and reversibility are central rather than incidental concerns, and that local deployment is feasible; yet no surveyed shell-level system extends beyond the command line to the full graphical session, and none has been evaluated with users against the conventional workflow it would displace.

2.5 Graphical and Computer-Using Agents

A parallel and larger body of work addresses computer-using agents[1] — systems that operate graphical environments by issuing actions and observing results. Although SynapseOS operates at the command line rather than by simulating graphical interaction, this literature is essential context: it establishes the dominant architectural patterns, defines the recurring technical bottleneck of visual grounding[2], and, through its platform coverage, reveals the gap that motivates the present study. The work is organized here by the platform each system targets — web, mobile, and desktop — together with the vision-language foundation models on which many depend.

Web Agents

The web has been the most heavily studied environment for language-guided agents. Mind2Web, introduced by Deng and colleagues [7], is the first dataset for developing generalist web agents that follow language instructions across diverse real-world websites, comprising over two thousand open-ended tasks collected from 137 websites across 31 domains; its accompanying two-stage model uses a small language model to filter candidate elements from large documents before invoking a larger model to choose among them. WebArena, introduced by Zhou and colleagues [8], provides a fully self-hosted, reproducible web environment of functional website clones and evaluates 812 long-horizon tasks for functional correctness rather than trajectory similarity, establishing a challenging benchmark on which contemporary models achieved completion rates far below human performance. Zheng and colleagues [9], in their SeeAct framework, decouple visual understanding from action: the model observes a screenshot, generates a textual plan, and a separate step grounds that plan onto page elements. Their central finding is diagnostic for the entire field — with oracle grounding provided, the model completes 51.1% of tasks on live websites, substantially outperforming a text-only baseline at 13.3%, but performance degrades sharply when grounding must itself be inferred, confirming visual grounding as the principal bottleneck.

Mobile Agents

Mobile-device control has produced large-scale datasets and progressively more autonomous agents. The pre-LLM work of Li and colleagues [19], discussed in §2.3, established the grounding problem for mobile interfaces. Android in the Wild, from Rawles and colleagues [10], contributes a dataset of 715,000 episodes spanning 30,000 unique instructions, in which actions are gestures at arbitrary screen coordinates rather than operations on indexed interface elements, exposing the challenge that actions must be inferred from visual appearance rather than from a structured model of the interface. AppAgent, from Zhang and colleagues [11], operates Android applications through a simplified action space of taps and swipes and learns to use new applications through autonomous exploration or by observing human demonstrations, generating application-specific documentation that matched or exceeded human-written documentation. Mobile-Agent, from Wang and

colleagues [12], adopts a purely vision-centric approach that requires no system metadata, using optical-character-recognition and icon-detection modules to locate interface elements — achieving broad applicability across mobile environments at the cost of vision-grounding errors that require recovery mechanisms.

Desktop and Cross-Application Agents

Desktop agents are the most direct graphical analogue to the present work, and their limitations are the most instructive. ScreenAgent, from Niu and colleagues [14], is a vision-language-model-driven agent for cross-application desktop control that operates in a plan–act–reflect cycle, specifying actions as coordinates and key sequences; its authors acknowledge that vision-only grounding is less reliable than approaches with API access. WinClick, from Hui and colleagues [15], targets Windows-desktop grounding and observes that most existing GUI agents rely on structured data formats such as the document object model or HyperText Markup Language, which are unavailable across general desktop environments; it therefore performs visual grounding directly from screenshots. UFO², the "Desktop AgentOS" of Zhang and colleagues [16], is the most ambitious desktop system surveyed: it fuses the native Windows accessibility framework with vision-based parsing in a hybrid action mechanism, dispatching subtasks from a coordinating agent to application-specialized agents. Its authors argue that existing computer-using agents are hindered by shallow operating-system integration, fragile screenshot-based interaction, and disruptive execution, and they explicitly identify Linux and macOS coverage, sandboxing for safety, and full-operating-system conversational replacement — as opposed to automation layered atop the operating system — as the next directions. This assessment, from within the most advanced desktop-agent project surveyed, corroborates the gap the present study addresses.

Vision-Language Foundation Models

Underlying many of these agents are vision-language models trained specifically for interface understanding. CogAgent, from Hong and colleagues [13], is an eighteen-billion-parameter visual language model specialized for GUI understanding and navigation, using both low- and high-resolution image encoders to recognize the small icons and text typical of graphical interfaces. It supports the position that pure-vision GUI agents are viable when the underlying model is trained appropriately — but at a compute cost, and with the cross-platform generalization limitations common to the entire graphical-agent literature. Across web, mobile, and desktop, these systems demonstrate broad capability within a single platform class, while consistently identifying visual grounding as an unsolved bottleneck and trending toward hybrid execution that prefers structured access where available and falls back to vision.

2.6 Accessibility and Speech-Driven Interaction

A distinct thread of the literature approaches conversational computing from the perspective of accessibility, targeting users with motor, speech, or cognitive needs. This thread is relevant to the present study in two ways: it provides the most rigorous human-centered evaluations in the field, and it exhibits a limitation that is

the mirror image of the mainstream agent literature, sharpening the opportunity the proposed system addresses.

VoicePilot, from Padmanabha and colleagues [5], is a framework for using LLMs as speech interfaces to physically assistive robots for individuals with motor impairments. Although its immediate target is robotic assistance rather than desktop computing, it is included here because it is the most rigorous human-centered evaluation in the LLM-speech-interface literature. The authors derive five concrete design guidelines from a study with motor-impaired participants, among them the importance of execution speed approaching that of a caregiver and the necessity of preview-and-confirm workflows — a guideline that directly informs the reversibility-based confirmation design of the present study. DexAssist, from Mehendale and Walishetti [6], targets web navigation by users with fine-motor impairments using two LLMs in tandem — one to interpret voice commands and a second to map the interpretation onto interface elements with fallback strategies — reducing the misinterpretation failures of single-model voice navigation on dense layouts. The architecture of van Dam [4] proposes retrofitting LLM-based conversational interfaces onto existing single-application graphical interfaces, exposing an application's navigation graph through a tool protocol and handling voice input through real-time speech recognition; its author observes that most production applications were never designed with speech in mind, making such retrofitting an ongoing challenge.

More recent work continues to mature the components required for speech-driven interface control. Han and colleagues, in GUIRoboTron-Speech [22], advance automated GUI agents driven directly by speech instructions, while Park and colleagues, in R-VLM [23], contribute region-aware visual grounding for more precise identification of on-screen elements. What unites this thread is that it operates at the level of the individual application or interaction technique rather than at the level of the session itself. These approaches presuppose an underlying conversational system surface on which to run, yet no such surface exists at the session level. This complementarity is significant: a general-purpose conversational session layer would serve not only mainstream users but also provide the substrate that accessibility-specific configurations require — positioning accessibility not as the immediate target of the present study, which scopes it out for recruitment and ethics reasons, but as a natural and well-grounded extension of its contribution.

2.7 Evaluation Environments and Benchmarks

The rapid proliferation of computer-using agents has been accompanied by increasingly rigorous evaluation environments, and these establish both the current ceiling of the field's capability and the methods by which that capability is measured. The most consequential is OSWorld, from Xie and colleagues [17], a benchmark of 369 real-world computer tasks in a real-computer environment spanning Ubuntu, Windows, and macOS. Its headline result is stark and frequently cited: humans achieve 72.36% success on these tasks, while the best evaluated model reaches only 12.24%. This wide gap, sustained across model families including GPT, Claude, Gemini, and Qwen, establishes that no current agent approaches human-level operating-system

competence, with weaknesses concentrated in GUI grounding and operational knowledge. OSWorld is also directly relevant to the present study as the source of a real-task suite against which benchmark comparability can be established.

Complementing the benchmark literature, the survey of LLM-brained GUI agents by Zhang and colleagues [18] synthesizes the field through 2024, organizing it around the components of perception, planning, action, and memory, and providing the four-layer vocabulary adopted in §2.2 of this review. It enumerates the field's open challenges — accurate element localization, knowledge retrieval, long-horizon planning, and safety-aware execution control — and identifies among the gaps it leaves open the empirical, full-operating-system conversational replacement that the present study pursues. What the evaluation literature reveals, however, is a specific methodological limitation: rigorous as these benchmarks are, they measure the autonomous task completion of agents acting on a user's behalf, not the comparative performance of a human user working through a conversational interface versus a conventional graphical one. That distinction motivates the evaluation design reviewed in §2.9.

2.8 Synthesis: Comparative Analysis and Research Gap

Positioning the surveyed works against one another along the four-layer pipeline makes the structure of the field, and the gap within it, explicit. Table 2.1 summarizes the principal systems by coverage scope, action mechanism, evaluation approach, and key limitation.

Table 2.1. Comparative analysis of surveyed systems on conversational and LLM-mediated interfaces for personal computing, positioned by coverage scope, action mechanism, evaluation, and key limitation.

Ref.	System	Year	Coverage scope	Action mechanism	Evaluation	Key limitation
[1]	Unix Consultant	1988	Shell consultation	Natural-language Q&A (no execution)	Demonstration, case studies	Hand-engineered knowledge base
[2]	NL2Bash	2025	Shell (CLI)	Command translation	600-pair test set, fine-tuning	Limited system-context awareness
[3]	NaSh	2025	Shell (Linux)	Guarded command translation	Worked examples, design paper	No formal user evaluation
[4]	Multimodal GUI Arch.	2025	Single application	API binding + voice	Architectural prototype	Per-application retrofitting
[5]	VoicePilot	2024	Assistive robot	Voice + robot API	User study, motor-impaired participants	Robot domain, not the OS

Ref.	System	Year	Coverage scope	Action mechanism	Evaluation	Key limitation
[6]	DexAssist	2024	Web (e-commerce)	Dual-LLM voice + element mapping	Web-task evaluation	Narrow to e-commerce
[7]	Mind2Web	2023	Web (137 sites)	Element selection from filtered HTML	Dataset benchmark	Structured-data-only in original form
[8]	WebArena	2024	Web (self-hosted)	Functional task execution	Benchmark, low model baseline	Simulated, not live web
[9]	SeeAct	2024	Web (live sites)	Visual plan + grounding	51.1% with oracle grounding	Grounding hard without oracle
[10]	Android in the Wild	2023	Mobile (Android)	Coordinate gestures	Dataset, 715k episodes	Brittle to interface updates
[11]	AppAgent	2023	Mobile (10 apps)	Tap/swipe + element labels	50-task evaluation	Per-app knowledge acquisition
[12]	Mobile-Agent	2024	Mobile (vision-only)	OCR + icon detection + planning	Mobile-Eval benchmark	Vision-grounding errors
[13]	CogAgent	2024	Cross-platform GUI	Vision-language model (high-res)	SOTA on VQA / navigation	Compute cost of high-res inputs
[14]	ScreenAgent	2024	Cross-app desktop	Vision + mouse/keyboard	Plan–act–reflect evaluation	Vision-only grounding fragility
[15]	WinClick	2025	Desktop (Windows)	Visual grounding	WinSpot benchmark	Windows-specific
[16]	UFO²	2025	Desktop (Windows)	Hybrid API + vision	System deployment, task benchmarks	Windows-only; automates atop the OS
[17]	OSWorld	2024	Ubuntu, Windows, macOS	Evaluation harness	369 tasks; 12.24% best vs 72.36% human	Reveals field-wide grounding gap
[19]	PixelHelp / seq2act	2020	Mobile (Pixel)	Two-stage transformer	ACL benchmark, three datasets	Pre-LLM; dataset-bound
[20]	ReAct	2023	Cross-domain	Thought–action–observation prompting	Multi-benchmark evaluation	Inherits LLM hallucination, brittleness
[22]	GUIRoboTron-Speech	2025	GUI (speech-driven)	Speech-instructed agent	Benchmark evaluation	Application, not session, level

Ref.	System	Year	Coverage scope	Action mechanism	Evaluation	Key limitation
[23]	R-VLM	2025	GUI grounding	Region-aware visual grounding	Grounding benchmark	Component, not full system
[25]	Local LLMs for NL2Bash	2026	Shell (CLI)	Open-source model evaluation	Large-scale model comparison	Evaluation, not a deployed system

Six patterns emerge from this comparison, each surfacing a recurring theme in the literature.

Bifurcation by Coverage Scope

The field is bifurcated by coverage scope. Shell-level work [2, 3] and single-application work [4, 5] sit at the narrow end; cross-application desktop, web, and mobile agents sit at the broad end but each within a single platform class. No surveyed system implements a working conversational replacement for the full graphical session that has been empirically validated against the conventional desktop workflow it would displace. The Unix Consultant [1] came closest in ambition four decades ago, but its pre-LLM foundation prevented it from scaling beyond demonstration.

Execution Mechanism at the Action Layer

A second pattern concerns the execution mechanism at the Action layer. The literature divides among command translation, API binding, and GUI simulation, with a dominant trend toward hybrid execution [16, 4] that prefers structured access where available and falls back to vision. Vision-only systems [12, 14] achieve broad applicability at the cost of grounding fragility, while structured-access systems achieve precision at the cost of per-environment integration. Notably, the Unix Consultant operated purely at the consultation layer without executing actions — an approach that reflects an underappreciated design principle, that the agent that explains can be safer than the agent that acts, which the confirmation-gated design of the present study revisits.

Visual Grounding as the Principal Bottleneck

A third pattern is that visual grounding is the field's principal bottleneck, identified explicitly by both SeeAct [9] and the GUI-agents survey [18]. The 51.1% ceiling of GPT-4V with oracle grounding, its sharp degradation without, and the 12.24% best-model performance on OSWorld [17] are all attributable in significant part to grounding failure. This bottleneck is a property of the graphical-simulation approach specifically; a command-line approach that grounds intent into shell commands rather than into on-screen pixel regions sidesteps it, at the cost of being bounded by what is expressible on the command line.

Scarcity of Comparative User Evaluation

A fourth pattern concerns evaluation method. Most surveyed systems are evaluated either through architectural demonstration [3, 4] or through curated task benchmarks [7, 8, 12, 17]. User studies are rare; only VoicePilot [5] conducts a substantive human-centered evaluation of LLM-based interaction, and no surveyed work compares its interface against the conventional graphical workflow for the same task on the same population. This is the empirical gap best filled by a controlled study of conversational interfaces.

The Operating-System Substrate

A fifth pattern concerns the operating-system substrate. Windows [15, 16], Android [10, 11, 12], and the web [7, 8, 9] are each heavily represented; Linux desktop coverage is conspicuously absent. NaSh [3] addresses Linux but only at the shell level, and OSWorld [17] includes Ubuntu but only as one of three evaluation platforms rather than as a primary deployment target. No surveyed system implements a working conversational session layer for the Linux desktop, despite Linux serving as the substrate for most developer and server-side workflows.

Shared Lineage and Its Fragility

A sixth pattern is the field's shared intellectual lineage and its attendant fragility. As established in §2.3, nearly every post-2023 system inherits the ReAct [20] convention of interleaving reasoning with action, which explains both the field's rapid progress and the pervasiveness of LLM-reasoning failure modes across it — a fragility that makes safety and recoverability mechanisms, rather than raw capability alone, decisive for any deployable system.

These patterns converge on a threefold research gap. *First*, no system in the literature replaces the conventional graphical session layer with a conversational one — that is, makes a conversational interface the primary surface a user encounters after login (replacing the user-facing desktop shell and panel) rather than an automation utility running on top of a standard desktop — whereas existing systems automate atop the desktop shell rather than substituting for it. Replacing the user-facing session layer in this manner preserves the underlying window manager and display server as implementation substrates (as detailed in the scope of Chapter 1) to avoid re-inventing basic desktop plumbing while offering a novel interface paradigm. *Second*, Linux desktop coverage is absent despite Linux being the substrate for most developer and server-side workflows. *Third*, no existing work compares a conversational interface against the conventional graphical workflow on the same tasks with the same user population. SynapseOS — a conversational session layer for the Linux desktop that translates natural-language intent into shell commands under a reversibility-based confirmation gate, evaluated against each participant's own operating system — is positioned precisely against these three gaps.

2.9 Review of the Methodological Literature

Beyond the systems it proposes to build upon, the present study draws on a methodological literature that shapes how a conversational interface should be evaluated. The evaluation practices of the surveyed work fall into three categories, each with a characteristic limitation. Architectural demonstrations [3, 4] establish feasibility but provide no performance evidence. Curated task benchmarks [7, 8, 17] provide reproducible, quantitative measures but assess the autonomous task completion of agents acting on a user's behalf rather than the performance of a human working through an interface. Human-centered user studies, exemplified by VoicePilot [5], provide the most direct evidence of an interface's value to real users but are rare in this literature and, where present, do not compare the proposed interface against the conventional workflow it would replace.

This critique motivates the methodological posture of the present study, which is grounded in established human-computer-interaction evaluation practice. The study adopts a *within-subjects* design, in which each participant completes tasks under both the SynapseOS condition and the condition of their own operating system, so that each participant serves as their own control and between-person variance is removed as a confound; the order of conditions is counterbalanced to distribute learning effects evenly. It further adopts an *expert-baseline* comparison — using each participant's most familiar environment as the comparison condition rather than a fixed alternative that may be unfamiliar — so that observed differences reflect interface quality rather than differential familiarity with the operating system, strengthening the ecological validity that the benchmark-only evaluations in the surveyed literature lack. To integrate the quantitative measures of task completion time and error rate with qualitative evidence of user experience, the study employs a *convergent mixed-methods* structure, collecting both concurrently and integrating them at the interpretation stage; the qualitative data, gathered in part through a think-aloud protocol, are analyzed using the thematic-analysis method of Braun and Clarke [24], a widely adopted six-phase approach for identifying and reporting patterns in qualitative data. Design considerations surfaced by the reviewed systems inform the evaluation directly: the preview-and-confirm guideline of VoicePilot [5] and the reversibility-and-guardrail framing of NaSh [3] together motivate an explicit validation of the confirmation gate and undo mechanism as part of the study.

The specific instantiation of this methodology — the composition of the benchmark and custom task suite, the participant profile and screening criteria, the session procedure, the metrics and their computation, and the statistical and qualitative analysis plan — is presented in detail in Chapter 3.

2.10 Summary

This chapter reviewed the literature on natural-language and LLM-mediated interfaces to personal computing across nearly four decades, organized along a four-layer pipeline of Input, Reasoning, Action, and Operating-System Integration. The historical foundations established the long-standing aspiration to a natural-language interface to a real operating system [1] and the methodological shift, driven by in-context reasoning [20], that made the current generation of systems possible. The shell-and-command-line thread most directly related to the proposed design demonstrated that natural-language-to-Bash translation is tractable [2], that safety and

reversibility are central concerns [3], and that local deployment is feasible [25], while leaving the extension from shell to session, and empirical user evaluation, as open work. The graphical-agent literature across web, mobile, and desktop [7–16] established the field's architectural patterns and its recurring bottleneck of visual grounding, while the accessibility thread [4, 5, 6, 22, 23] revealed a complementary limitation that a general conversational session layer would resolve. The evaluation literature [17, 18] quantified the wide gap between current agents and human competence and exposed a methodological gap: interfaces are not evaluated comparatively against the conventional workflow with real users.

Synthesized, the surveyed works reveal a threefold gap: no conversational replacement for the graphical session layer, no coverage of the Linux desktop, and no controlled comparison of a conversational interface against the conventional workflow across user populations. The present study addresses all three through SynapseOS, a conversational session layer for the Linux desktop evaluated by controlled, within-subjects comparison against each participant's own operating system. The methodology through which this evaluation is conducted is the subject of Chapter 3.

Notes

1. A computer-using agent is an artificial-intelligence system that operates a computing environment by issuing actions and observing their results. A GUI agent is a computer-using agent whose action space consists of operations on a graphical user interface — typically clicks, key presses, scrolls, and text entry — usually driven by a large language model or a multimodal (vision-language) model that additionally accepts images as input.
2. Grounding is the problem of mapping a model's textual plan onto a concrete operation in the environment. Visual grounding is the specific subproblem of identifying the on-screen pixel region corresponding to a referenced interface element — repeatedly identified in the surveyed literature as the principal bottleneck for graphical agents, and one that a command-line approach avoids by grounding intent into shell commands rather than into screen coordinates.

Chapter 3

METHODOLOGY

This chapter describes the research methodology employed in designing, implementing, and evaluating SynapseOS — a system that replaces the conventional graphical session layer with a conversational interface layer. The methodology is organized into three sections: Dataset, which defines the data to be collected and how it will be gathered; Procedure, which describes the step-by-step process of building the system and conducting the study; and System Testing, which defines the evaluation design, metrics, and statistical approach used to compare SynapseOS against the conventional Linux desktop.

This methodology directly answers the three research questions posed in Chapter 1: *RQ1*, how a conversational session layer can be designed as the primary interface to a Linux-based personal computing environment; *RQ2*, what natural-language-to-bash intent parsing and execution design — incorporating a reversibility-based confirmation gate and an undo mechanism — most effectively grounds natural language intent into safe, recoverable system actions; and *RQ3*, how SynapseOS compares to each participant's own primary operating system in task completion time, error rate, and user satisfaction across novice and power user populations. Section 1 defines the data gathered to answer RQ2 and RQ3; Section 2 describes the system built to answer RQ1 and RQ2; and Section 3 defines the evaluation design used to answer all three.

Conceptual Framework

SynapseOS operates as a conversational shell layer positioned between the user and the Linux command-line environment. The user issues natural language commands through a persistent chat interface. An intent parsing pipeline — powered by a locally-hosted small language model — translates each utterance into a shell command, which is executed in a bash subprocess. Output is captured and rendered back in the conversational interface. The user issues intent in natural language and receives results in natural language; the shell command is an implementation detail invisible to the user. This architecture is illustrated in Figure 3.1.

The framework is evaluated through a controlled user study in which participants complete a fixed set of tasks under two conditions: using SynapseOS as the interface, and using their own primary operating system — Windows, macOS, or Linux — as the baseline. This expert baseline design pits SynapseOS against the environment each participant already knows and uses daily, generating empirical data on task completion time, error rate, and user satisfaction across the real-world OS landscape [17].

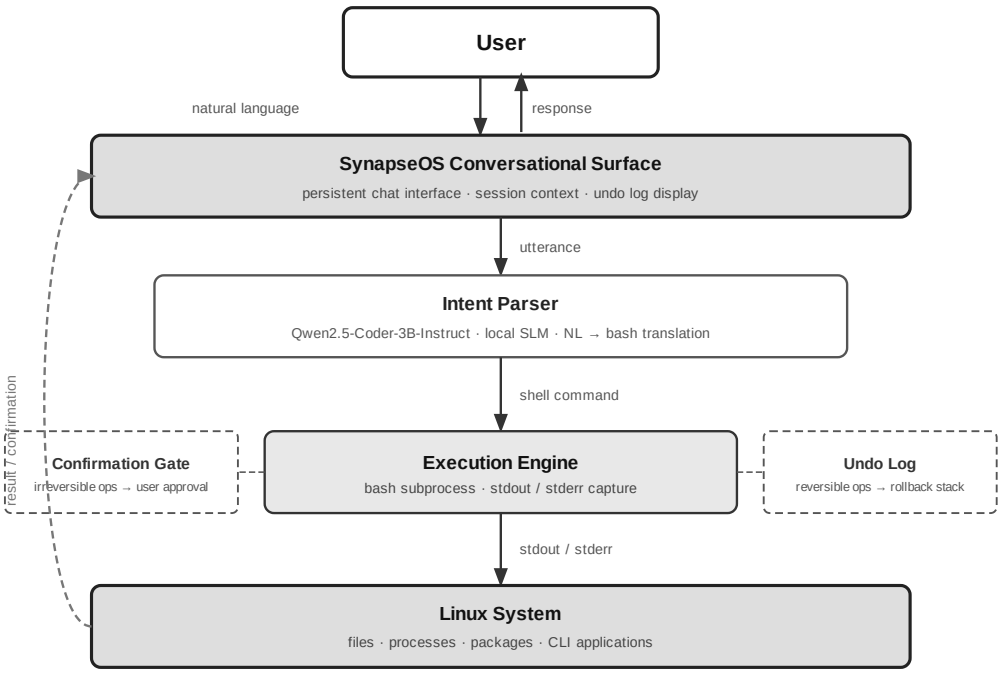

Figure 3.1. Conceptual framework of SynapseOS. The user issues a natural language command to the conversational surface, which routes the utterance to the intent parser (Qwen2.5-Coder-3B-Instruct, local SLM). The intent parser produces a shell command dispatched to the execution engine (bash subprocess). Irreversible operations are intercepted by the confirmation gate; reversible operations are logged to the undo stack. Command output (stdout/stderr) is captured, rendered in the conversational surface, and presented back to the user — completing the interaction loop.

System Environment Specifications

The user study is conducted across three dedicated machines — one running SynapseOS (Condition A) and two covering the three supported baseline operating systems (Condition B). Table 3.1 describes each machine's role and operating environment. Exact processor, RAM, and storage specifications for each machine are confirmed and documented in the study protocol prior to the pilot study. Table 3.2 specifies the minimum hardware and software requirements for deploying SynapseOS independently of the study configuration.

Table 3.1. Study evaluation machines

Machine	Role	OS	Notes
SynapseOS evaluation machine	Condition A — all participants	SynapseOS	GUI mode; fullscreen conversational interface running within an XFCE (X11 session) desktop environment; based on Debian 13 (Trixie); Ollama + Qwen2.5-Coder-3B-Instruct running locally; a participant-accessible fallback returns to the underlying XFCE session if needed, logged and excluded from the primary analysis (Section 3.5)
Windows / Linux dual-boot baseline	Condition B — Windows- or Linux-primary	Windows 11 or Ubuntu 24.04 LTS (GNOME),	Standard consumer x86-64 hardware; Windows 11 and Ubuntu installed on separate partitions; standard user account per OS; no custom

machine	participants	selected at boot	software. Facilitator boots to the OS matching the participant's assignment before the session begins.
macOS baseline machine	Condition B — macOS-primary participants	macOS (current release)	Apple Silicon hardware; standard user account

Table 3.2. SynapseOS minimum deployment requirements. SynapseOS is distributed as a Linux system based on Debian 13 (Trixie).

Component	Minimum Requirement
Processor	64-bit x86 processor, 2-core minimum; no GPU required
RAM	4 GB (CLI mode[8] / TUI mode[1]); 8 GB recommended (GUI mode)
Storage	10 GB available (operating system, language model, and application)
Display	CLI mode / TUI mode: any terminal or remote connection; no graphical environment required. GUI mode: graphical desktop environment required.
Language Model	Locally-hosted small language model (~1.8 GB)[2]; CPU-only inference; no GPU required. Cloud language model backend available as an optional, user-configured alternative.
Inference Server	Local inference server (Ollama)[3]; manages model lifecycle, loading, and query serving on the host machine.

The user study evaluates SynapseOS in GUI mode, which requires a graphical desktop environment and 8 GB RAM. If SynapseOS becomes unresponsive during a task or a participant wishes to stop using it, a fallback returns them to the underlying XFCE session, which continues running beneath the conversational interface throughout Condition A; every fallback invocation is logged, and any task during which it occurs is excluded from the primary completion-time and error-rate comparison rather than counted as a SynapseOS success (Section 3.5). SynapseOS additionally supports a TUI mode — a terminal-based interface requiring no display server, running on as little as 2 vCPU and 4 GB RAM — designed for server and remote deployment environments accessible over SSH, and a CLI mode[8] — a one-shot, non-interactive invocation that translates a single natural-language request into a proposed command, passes it through the same reversibility-based confirmation gate described in the Execution Pipeline (Section 2.1) — executing immediately if reversible, blocking pending explicit confirmation if not — and exits once execution completes, sharing TUI mode's minimal hardware footprint but intended for scripting and automation rather than an interactive session. Neither CLI nor TUI mode is evaluated in the user study.

Section 1: Dataset

The data gathered for this study falls into three categories: benchmark task data used to evaluate the system in a standardized environment, user study data collected from human participants in a controlled setting, and system performance data captured automatically through built-in telemetry during all sessions.

1.1 What to Gather

a) Benchmark Task Suite

The primary benchmark dataset is the OSWorld task suite [17], which provides standardized real-computer tasks across shell operations, file management, web browsing, and application-level interactions. OSWorld is the only existing benchmark that evaluates computer-using agents in real operating system environments, making it the most appropriate external standard for SynapseOS evaluation. Tasks are presented as natural language instructions; agents interact with a real virtual machine environment, and outcomes are evaluated by automated inspection of the resulting system state — whether the goal was actually achieved — rather than by comparing individual actions. In addition to OSWorld tasks, a custom cross-platform task suite is developed for this study, covering file search and organization, system and process monitoring, application and package management, and text and data processing — each task defined as a human-level goal achievable via SynapseOS natural language commands and via native OS tools on the comparison machine. Each task in both suites is documented with a description, expected outcome, task category, and difficulty level.

b) User Study Data

The following data types are collected from each participant during the user study:

- **Task completion time:** the elapsed time from when the task prompt is presented to when the participant signals task completion or the session logs a terminal event
- **Error rate:** the count of failed tasks, incorrect outcomes, and recovery attempts per participant per condition
- **User satisfaction scores:** post-study responses to the System Usability Scale (SUS) and the NASA Task Load Index (NASA-TLX), both validated instruments for measuring perceived usability and cognitive workload respectively
- **Qualitative feedback:** responses gathered through a semi-structured post-study interview covering perceived ease of use, trust in the system, and comparison between the two conditions

c) System Performance Data

The following telemetry data is captured automatically by SynapseOS throughout all sessions:

- Command execution latency: time elapsed from utterance submission to bash subprocess completion
- Undo log events: count and type of operations logged to the undo stack
- Confirmation prompts triggered: count of irreversible operations that required user confirmation
- Conversational turns per task: the number of utterances required to complete each task
- Intent parsing accuracy: whether the system correctly identified the user's intent on the first attempt

1.2 How to Gather

a) Benchmark Tasks

SynapseOS is deployed on the Debian 13 (Trixie) environment matching the OSWorld evaluation setup [17]. Each benchmark task is executed through the conversational interface, and results are captured programmatically through the built-in telemetry system. Tasks are run in automated sessions where SynapseOS is given the task prompt as a natural language instruction and the system's output is compared against the expected outcome defined in the benchmark. OSWorld evaluates autonomous agents by inspecting the resulting system state after independent task execution, not a human operator's performance; SynapseOS is the study's only autonomous agent, so it is the only system evaluated against this benchmark. Condition B has no autonomous-agent equivalent — a human participant operates it directly — and is instead evaluated through the custom cross-platform task suite administered in the user study (Section 1.2b).

b) User Study

The user study is conducted in a controlled laboratory setting with two machines: the SynapseOS evaluation machine (Debian 13, conversational interface) and the baseline machine running the participant's native OS — Windows 11, macOS, or Linux with a GNOME desktop — as determined during screening. Each participant completes the full task set on both machines under a within-subjects design, with condition order counterbalanced across participants to mitigate learning and carryover effects. Specifically, half of the participants complete the SynapseOS condition first, and the remaining half complete the native OS condition first. Screen capture software records activity on both machines; the SynapseOS session logger supplements the recording for the SynapseOS condition.

The study involves 20 participants divided into two groups:

- **Novice users (n = 10):** participants with limited computing experience, capable only of basic tasks (web browsing, document editing) on any OS, and no prior command-line familiarity, recruited from the general university population
- **Power users (n = 10):** participants with demonstrated proficiency in computing on their primary OS — capable of performing file management, system monitoring, and process administration tasks efficiently — recruited from the university's information technology and computer science programs

Participants are recruited through university channels. Recruitment enforces a minimum quota of two participants per primary OS background (Windows, macOS, Linux), so that the realized sample cannot end up concentrated in a single OS and leave the per-OS exploratory subgroup analysis (Section 3.2) without data for the others. Informed consent is obtained from all participants prior to the session. Ethics approval is secured before recruitment begins. Participant data is anonymized at the point of collection.

After completing each condition, participants fill out the SUS and NASA-TLX questionnaires. After completing both conditions, a brief semi-structured interview is conducted to gather qualitative feedback.

1.4 Why

The OSWorld benchmark is used as the primary evaluation standard because it is the only existing benchmark that evaluates computer-using agents in real operating system environments with real applications [17]. Its use allows SynapseOS results to be situated within the existing literature.

A custom cross-platform task suite is necessary because OSWorld's task coverage does not represent the human-goal-framed system tasks relevant to this study — file organization, system monitoring, process management, and text processing — achievable both through SynapseOS natural language commands and through participants' native OS tools (Windows, macOS, or Linux graphical desktop). Tasks are scoped to operations achievable via bash on SynapseOS and via native graphical or terminal tools on the comparison machine.

The evaluation follows a convergent (parallel) mixed-methods design: the quantitative performance measures — task completion time, error rate, SUS, and NASA-TLX — and the qualitative interview data are collected concurrently within each session and integrated at the interpretation stage, so that the measured differences between conditions and participants' accounts of why those differences arise inform one another rather than being read in isolation.

The within-subjects design is chosen because it allows each participant to serve as their own control, producing a direct comparison between conditions while requiring a smaller participant pool than a between-subjects design. Counterbalancing controls for the primary threat to internal validity: the learning effect, wherein participants may perform better in the second condition simply due to familiarity with the tasks.

The expert baseline design — using each participant's primary OS as Condition B rather than a fixed Linux graphical desktop — is chosen because a fixed Linux baseline would confound interface quality with OS familiarity. The overwhelming majority of participants have never used a Linux desktop; performance under that condition would reflect unfamiliarity with a foreign environment, not a meaningful comparison of interface paradigms. The stronger and more ecologically valid test is whether SynapseOS can compete with what participants already know and use daily. A positive result against a participant's native environment is a more compelling argument for SynapseOS than a win against a system participants are encountering for the first time.

The dual-population design — novice and power users — directly addresses the gap identified in Chapter 1, where no prior work has evaluated a conversational interface across user populations with differing technical backgrounds [17]. The three data categories align with the three research questions: RQ1 is addressed by the system design and benchmark task coverage; RQ2 is addressed by system performance telemetry (command execution latency, intent parsing accuracy, undo and confirmation events); and RQ3 is addressed by the user study metrics (task completion time, error rate, satisfaction scores).

1.5 Participant Screening Criteria

Participants are assigned to the novice or power user group based on a pre-study screening questionnaire administered before the session begins. The questionnaire assesses the following dimensions:

- Years of general computer use
- Primary OS — the operating system used as the primary daily computing environment (Windows / macOS / Linux); must average four or more hours of use per day to qualify
- Self-reported proficiency on primary OS (basic / intermediate / advanced)
- Self-reported command-line familiarity (none / basic / intermediate / advanced)
- Prior exposure to conversational AI interfaces — frequency of use (never / occasionally / regularly / daily) and which tools (chatbots, voice assistants, code-completion or agentic coding assistants, other LLM-based tools)

Self-reported proficiency is confirmed with a short behavioral screener: three timed mini-tasks performed unaided on the participant's primary OS using its native tools, one per system-task category used in the main study — file organization (locate and relocate a set of recently modified files), process monitoring (identify the application consuming the most memory via the OS's native system monitor), and application management (determine whether a named application is installed and identify its version). Each task is scored pass/fail by the facilitator against a fixed time limit and completion criterion. The behavioral screener exists because self-reported proficiency alone is not a reliable classifier — a participant's technical background in one domain (e.g., software development) does not guarantee fluency with a given OS's native file management, process monitoring, and system administration tools, which is the specific competency this study's novice/power-user split is intended to capture.

Participants are classified as **novice users** if they report basic proficiency on their primary OS, computing use limited to web browsing and document editing, no prior command-line experience on any platform, and fail the behavioral screener. Participants are classified as **power users** if they report advanced proficiency on their primary OS — capable of performing file management, process monitoring, and system administration tasks efficiently using that OS's native tools — regardless of which OS that is, and pass all three behavioral screener tasks. Participants whose self-report and behavioral screener results disagree, or who do not clearly satisfy either classification, are excluded. Primary OS is recorded for all participants as a grouping variable, and prior conversational AI exposure as a covariate, in the analysis; prior AI exposure is analyzed as an exploratory covariate only (see Section 3.2) rather than a primary grouping variable, since splitting the n = 20 sample along a third axis would leave subgroups too small to support a hypothesis test.

1.6 Task Design

The custom cross-platform task suite is designed to cover human-level system tasks achievable both through SynapseOS natural language commands and through native tools on the participant's primary OS. Task design follows four principles:

1. **Ecological validity** — tasks reflect actions a real user would perform in their everyday computing environment: finding and organizing files, monitoring system resources, managing processes, and handling text data

2. **Cross-platform equivalence** — all tasks are defined as human-level goals achievable both through SynapseOS natural language commands (mapped to bash) and through native tools on the participant's primary OS (File Explorer, Task Manager, Finder, Activity Monitor, GNOME, or equivalent). The method of achievement differs by platform; the human goal does not
3. **Difficulty stratification** — tasks are rated at three difficulty levels (simple, moderate, complex) based on the number of steps required and the specificity of the target operation
4. **Population appropriateness** — both participant groups receive the same task set, with task wording written in plain language accessible to novice users and free of OS-specific or CLI-specific terminology

The custom task suite comprises tasks organized into four categories: file search and organization, system and process monitoring, application and package management, and text and data processing. All tasks are defined at the human-goal level — achievable via SynapseOS natural language on the evaluation machine and via native graphical or terminal tools on the comparison machine. Each task is documented with a unique identifier, a natural language prompt as presented to the participant, the expected outcome, and the assigned difficulty level. Tasks are validated during the pilot study and revised as necessary before the main study begins.

Section 2: Procedure / Experiment

Building SynapseOS and running the study that evaluates it are two related but distinct efforts. The system itself — the conversational interface layer and execution pipeline — is designed and implemented first, followed by the scaffolding that makes the resulting study trustworthy: collected data is processed and anonymized, the system and instruments are validated, each participant session follows a standardized procedure, and participant welfare is protected throughout.

2.1 System Design and Implementation

The construction of SynapseOS proceeds across two primary workstreams — the conversational interface layer and the execution pipeline — integrated into a unified session manager that launches in place of the conventional Linux graphical desktop.

a) Conversational Interface Layer

The conversational surface is implemented as a persistent, full-screen chat interface that serves as the primary and sole user-facing interface when SynapseOS is active as the session manager. The interface accepts natural language input, displays system responses and command output, and maintains conversational context across the session. An intent parsing pipeline translates each user utterance into a shell command for execution.

Conversational context is session-scoped: history is held in memory for the duration of the login session and is cleared on logout or reboot, with no persistence layer in the thesis prototype. When accumulated history approaches 75% of the intent parsing model's 8,000-token context limit, a rolling window discards the oldest turns

to keep the conversation within budget. Verbose command output — such as a full directory listing or search result — is truncated to a compact summary before being appended to history, preventing a single command's output from consuming a disproportionate share of the available context.

SynapseOS is designed around a locally-hosted small language model (SLM) as the default, model-agnostic inference backend, with cloud inference available only as an explicit opt-in. Because the system operates at the OS level — processing every natural language command and observing terminal output, file paths, command history, and interaction patterns — routing inference through a cloud API would give an external provider continuous visibility into the user's computing activity. On-device inference eliminates this exposure by default; a user may still supply an API key for a supported cloud provider to substitute a hosted frontier model, entirely at their discretion. The default configuration requires no API key, no internet connection, and no per-query cost, making SynapseOS fully operational in offline and air-gapped environments.

The primary SLM is Qwen2.5-Coder-3B-Instruct, an open-weights[4] code-specialized model that demonstrated the strongest natural language-to-bash translation accuracy among models below 7B parameters in controlled evaluation [2]. Among the models directly compared in that evaluation, its code-specialized pretraining produced a clear margin over general-purpose models of comparable or larger size — 58% NL2Bash accuracy with prompt engineering, versus 49% for Llama 3.2-3B and 37% for a LoRA-fine-tuned Llama 3.2-1B — attributable to Qwen2.5-Coder's training corpus of 5.5 trillion code tokens against Llama 3.2's general-purpose pretraining. At 4-bit quantization[5], the model requires approximately 1.8 GB of memory and runs on the target Debian 13 (Trixie) hardware without a dedicated GPU. The model is fine-tuned via Low-Rank Adaptation (LoRA)[6] on the NL2Bash corpus[7] and a custom SynapseOS task suite to maximize intent parsing accuracy.

Accuracy is measured using an execution-based scoring methodology rather than syntax-level string matching, following Westenfelder et al. [2] — see Section 2.3 for the full evaluation procedure.

This selection was reconfirmed against models released after the initial literature review, to guard against the fast-moving small-model landscape becoming stale before study execution; no smaller alternative offered comparable shell-generation evidence within the same CPU-only hardware budget [25]. Newer mixture-of-experts variants require all experts resident in memory regardless of active parameters per token, incompatible with this study's memory budget, and newer general-purpose small models report reasoning benchmarks but no shell-command generation evidence.

b) Execution Pipeline

Once the intent parser produces a shell command, SynapseOS passes it through two safety mechanisms before execution, following the reversibility-and-guardrail framing established for LLM-powered natural language shells [3] and the preview-and-confirm design guideline validated with motor-impaired users of an LLM-mediated assistive-robotics interface [5]. First, the command is classified by reversibility: operations that are irreversible — such as file deletion, system configuration changes, or destructive package operations — trigger a confirmation prompt requiring explicit user approval before execution proceeds. Reversible operations are logged to the undo

stack, enabling the user to request rollback at any point through the conversational interface. Second, the approved command is dispatched to a bash subprocess. Standard output and standard error are captured and rendered in the conversational surface as the system's response to the original natural language input.

SynapseOS executes only shell-expressible operations. The system's capability boundary is the Linux command-line toolchain: file management, process control, text processing, package management, network configuration, and any application that exposes a command-line interface. GUI-only interactions — clicking buttons in graphical applications, navigating web page elements, or interacting with visual-only interfaces — are explicitly outside scope, in deliberate contrast to broader desktop agent frameworks that additionally orchestrate GUI automation atop the existing session [16]. This boundary is intentional: it constrains the research to a tractable, verifiable claim that the system can be built, evaluated, and defended within the thesis timeline.

2.2 Data Pre-processing

Prior to statistical analysis, all data collected during the benchmark evaluation and user study — not the system itself — undergoes the following processing steps:

- **Task normalization:** task descriptions and expected outcomes are standardized across both the OSWorld benchmark and the custom task suite to ensure consistent evaluation criteria
- **Log parsing and event extraction:** raw session logs from SynapseOS telemetry are parsed to extract per-task metrics — completion time, error events, and undo events
- **Metric computation:** task completion time is computed as the elapsed time between task prompt presentation and task completion event; error rate is computed as the ratio of failed or incorrect task outcomes to total task attempts per participant per condition; SUS and NASA-TLX scores are computed according to their respective standard scoring formulas
- **Participant data anonymization:** all participant records are anonymized at the point of extraction, replacing identifying information with participant ID codes prior to any analysis

2.3 Validation

The following validation procedures are applied to ensure the reliability and accuracy of the system and the study instruments:

- **Pilot study:** conducted with five participants prior to the main study to validate the task set difficulty, the questionnaire instruments, and the session procedure; findings from the pilot study are used to revise instruments and procedures before the main data collection begins
- **Intent parsing accuracy:** evaluated using an execution-based scoring methodology [2] — each generated command is executed in a controlled environment and its resulting system state compared against a human-annotated expected outcome for each task utterance, rather than syntax-level string matching, with ambiguous cases resolved through LLM-assisted functional equivalence assessment at 95% confidence; accuracy is computed as the proportion of first-attempt matches

- **Confirmation policy effectiveness:** evaluated by verifying that all irreversible operations in the task set trigger the confirmation prompt as designed, and that no irreversible operations execute without user approval
- **Undo success rate:** evaluated by executing a representative set of reversible operations and verifying that the undo log correctly restores the prior system state in each case

2.4 Study Session Procedure

Each participant session follows a standardized procedure conducted in the following order:

1. **Briefing:** The session facilitator explains the study purpose, session structure, and participant rights. The participant reads and signs the informed consent form. The pre-study screening questionnaire is administered to confirm group classification.
2. **System familiarization:** For the SynapseOS condition, the participant is given a five-minute orientation to the conversational interface and shown how to enter natural language commands. For the native OS condition, no familiarization is provided — participants use their home environment as they normally would. No task-specific training is provided under either condition.
3. **Task completion:** The participant completes the full task set under the assigned condition. Tasks are presented one at a time via a printed task prompt sheet. The session logger captures start and end times automatically. Screen recording captures all on-screen activity. The participant is instructed to think aloud throughout.
4. **Post-condition questionnaire:** Immediately after completing all tasks under a condition, the participant completes the SUS and NASA-TLX questionnaires for that condition.
5. **Condition changeover:** If the participant has not yet completed both conditions, a short break is offered, the participant moves to the other machine (SynapseOS or native OS), and the session returns to step 2.
6. **Post-study interview:** After both conditions are completed, the facilitator conducts a semi-structured interview of approximately 10–15 minutes covering perceived ease of use, trust in the conversational interface, preference between conditions, and open-ended feedback.
7. **Debrief:** The facilitator discloses the full study objectives, answers participant questions, and thanks the participant. Session data is anonymized immediately upon session close.

   Total estimated session duration per participant: 90–120 minutes.

2.5 Ethical Considerations

This study involves human participants and is subject to ethics review and approval by the Institutional Review Board (IRB) or equivalent ethics committee of Mapúa University – Makati prior to the commencement of participant recruitment. The following ethical principles are observed throughout the study:

- **Informed consent:** All participants receive a written information sheet describing the study purpose, procedures, data collected, and their rights. Participation is entirely voluntary; participants may withdraw at any time without penalty or consequence.
- **Anonymization:** Participant identities are not recorded. Each participant is assigned an anonymous ID code at the point of enrollment. All collected data — session logs, questionnaire responses, and interview transcripts — are stored and analyzed using participant ID codes only.
- **Data minimization:** Only the data types specified in Section 1.1 are collected. No identifying metadata — names, email addresses, or device identifiers — is retained beyond the consent confirmation step.
- **Data security:** All collected data is stored on password-protected institutional infrastructure accessible only to the research team. Interview recordings are deleted after transcription and verification.
- **Confidentiality:** Study findings are reported in aggregate. No individual participant's data is disclosed in any form that could enable identification.
- **Right to withdraw:** Participants may discontinue the session and request withdrawal of their data at any point during or after the session, up to the point at which anonymization makes individual data unidentifiable.

Section 3: System Testing

SynapseOS is evaluated against the datasets and study design established in Section 1. The comparison follows a fixed set of metrics and formulas, a statistical analysis plan for testing significance and effect size, and an accounting of the threats to validity this design addresses.

3.1 Evaluation Dataset

The system is evaluated against two datasets: the OSWorld benchmark task suite [17] for direct comparison with results in the existing literature, and the custom cross-platform task suite described in Sections 1.1 and 1.6. The user study evaluation involves 20 participants (10 novice, 10 power users); the baseline condition is each participant's primary operating system — Windows 11, macOS, or Linux with GNOME desktop — on a dedicated baseline machine.

3.2 Evaluation Profile

The study employs a within-subjects design with two conditions:

- **Condition A:** SynapseOS (conversational interface layer on the Debian 13 evaluation machine)
- **Condition B:** Participant's primary operating system (Windows 11, macOS, or Linux with GNOME desktop, as determined by pre-study screening, on the dedicated baseline machine)

All 20 participants complete both conditions. Condition order is counterbalanced: 10 participants complete Condition A first, and 10 participants complete Condition B first. This counterbalancing controls for order effects and task familiarity.

The two participant groups — novice users and power users — are analyzed both in aggregate and separately to determine whether the effect of the interface condition differs across experience levels. Additionally, primary OS (Windows, macOS, Linux) is recorded as a secondary grouping variable to determine whether results vary across participant backgrounds, and prior conversational AI exposure (frequency of use) is recorded as a covariate to determine whether familiarity with LLM-based tools correlates with SynapseOS performance independent of OS proficiency. The primary analysis is the pooled within-subjects comparison across all 20 participants; per-OS subgroup analyses and AI-exposure correlations are treated as exploratory given the smaller per-group sample sizes expected at n = 20.

3.3 Criteria and Formula

The following metrics and formulas are used to evaluate SynapseOS against the baseline:

a) Task Completion Time

Mean task completion time is computed per condition per participant group. Inferential testing and effect size reporting follow the procedures described in Section 3.4.

b) Error Rate

Error rate is computed as the proportion of task attempts resulting in failure or incorrect outcome:

$$\textit{Error Rate} = (\textit{Failed Tasks} + \textit{Incorrect Outcomes}) / \textit{Total Task Attempts}$$

Error events are further categorized by type: intent parsing errors (incorrect bash command generated), execution errors (bash subprocess failure), and recovery failures (unsuccessful undo or retry). The same paired statistical test is applied to error rate differences between conditions.

c) User Satisfaction

SUS scores are computed per the standard scoring procedure to produce a score on a 0–100 scale; a score above 68 is considered above-average usability. NASA-TLX scores are computed as the unweighted mean of the six subscale ratings to produce a composite workload score per participant per condition.

Qualitative interview data is analyzed using thematic analysis to identify recurring themes in participant perceptions of both conditions.

d) Statistical Significance and Effect Size

All primary comparisons between conditions are tested at $\alpha = 0.05$. Effect size is reported using Cohen's *d*, with thresholds of 0.2 (small), 0.5 (medium), and 0.8 (large). Minimum sample size justification is based on an a priori power analysis targeting 80% statistical power at the specified significance level and a medium effect size

(d = 0.5), which yields a minimum of 17 participants for a within-subjects design — satisfied by the 20-participant sample.

3.4 Data Analysis Plan

Quantitative analysis proceeds in three stages. First, descriptive statistics — means, standard deviations, and 95% confidence intervals — are computed for all primary metrics (task completion time, error rate, SUS score, NASA-TLX score) per condition and per participant group. Second, the Shapiro-Wilk test is applied to assess normality of each distribution before selecting the appropriate inferential test: the paired-samples t-test for normally distributed data, or the Wilcoxon signed-rank test for non-normally distributed data. Third, effect size (Cohen's d) is computed for all statistically significant comparisons.

Subgroup analysis is conducted separately for the novice and power user groups to determine whether the effect of the interface condition differs across populations. The interaction between interface condition and user group is examined to assess whether SynapseOS produces differential effects depending on technical background.

Qualitative data from post-study interviews is analyzed using reflexive thematic analysis following the six-phase procedure described by Braun and Clarke [24]: familiarization with the data, initial code generation, theme search, theme review, theme definition, and write-up. Two members of the research team independently code a random subset of transcripts; inter-rater agreement is assessed and disagreements are resolved through discussion before finalizing the codebook. Themes are reported alongside representative participant quotations.

3.5 Threats to Validity

The following threats to validity are identified and the mitigations applied in this study are described.

Internal validity:

- **Learning effect:** Participants may perform better on the second condition simply due to task familiarity, independent of interface quality. *Mitigation:* condition order is counterbalanced across participants so that learning effects are distributed equally across both conditions and do not systematically favor either.
- **Demand characteristics:** Participants may behave differently from natural use because they know they are being observed. *Mitigation:* participants are instructed to complete tasks as they naturally would; the specific hypotheses of the study are withheld until the post-study debrief to reduce social desirability bias.
- **Fatigue:** Completing the full task set twice in a single session may degrade performance in the second condition independent of interface quality. *Mitigation:* a short break is offered between conditions and total session length is bounded to approximately 120 minutes.
- **Fallback reliance:** GUI mode provides a participant-accessible fallback to the underlying XFCE session if SynapseOS becomes unresponsive or a participant wishes to stop using it mid-task; unrestricted, this would let a participant's own choice substitute for the interface under evaluation, confounding the within-

subjects comparison. *Mitigation:* every fallback invocation is logged with participant ID, task ID, and timestamp; any task during which it is invoked is excluded from the primary completion-time and error-rate comparison for that task rather than counted as a SynapseOS success, and fallback-invocation rate is reported separately as an exploratory reliability metric.

External validity:

- **Sample generalizability:** The study uses a university-recruited sample of 20 participants across three primary OS backgrounds (Windows, macOS, Linux), which limits direct generalization to the broader population of computer users. Findings should be interpreted as indicative rather than definitive; replication with larger and more diverse samples is recommended as a direction for future work.
- **Task generalizability:** The evaluation is bounded by OSWorld coverage and the custom task suite developed for this study. Computing workflows outside this coverage are not evaluated, and results may not extend to all possible use cases.
- **Platform specificity:** SynapseOS is evaluated on a single hardware and software configuration — a dedicated machine running SynapseOS in GUI mode (XFCE desktop environment, X11 session) on a Debian 13 (Trixie) base. Performance characteristics on different hardware, display configurations, or Linux distributions may differ from the reported results.

Notes

1. Terminal User Interface: a text-based interactive interface that runs in a command-line terminal. TUI mode requires no graphical display server or desktop environment and can be accessed remotely over SSH.
2. The SynapseOS prototype uses Qwen2.5-Coder-3B-Instruct, a code-specialized open-weights model selected for its NL2Bash accuracy. See Section 2.1a for selection rationale and benchmark evidence.
3. An inference server loads a language model into memory and processes query requests from the application layer. Ollama is an open-source tool for running language models locally on consumer hardware without cloud connectivity.
4. Open-weights models make their trained parameters publicly available for local download and use; users run inference on their own hardware without routing queries to an external service provider.
5. Quantization reduces the numerical precision of model weights to lower memory requirements. At 4-bit precision, a model that would otherwise require dedicated GPU memory can run on a standard CPU with modest RAM.
6. Low-Rank Adaptation (LoRA) is a parameter-efficient fine-tuning method that introduces a small number of trainable weight matrices while keeping the base model frozen, enabling task-specific adaptation on consumer hardware without full retraining.
7. NL2Bash is a benchmark dataset pairing natural language command descriptions with their corresponding bash shell commands, used to evaluate a model's accuracy in translating natural language into executable commands.
8. Command-Line Interface mode: a one-shot, non-interactive invocation that translates a single natural-language request into a proposed command, runs it through the same reversibility classifier and confirmation gate as the other interface modes, executes it (immediately if reversible, after confirmation if not), and exits, with no persistent session. Distinct from TUI mode's persistent interactive chat session; intended for scripting, automation, and one-off remote commands.

REFERENCES

[1] Wilensky, R., Chin, D. N., Luria, M., Martin, J., Mayfield, J., & Wu, D. (1988). The Berkeley Unix Consultant Project. *Computational Linguistics*, 14(4), 35–84. https://aclanthology.org/J88-4003/

[2] Westenfelder, F., Hemberg, E., Tulla, M., Moskal, S., O'Reilly, U.-M., & Chiricescu, S. (2025). LLM-Supported Natural Language to Bash Translation. *arXiv:2502.06858*. https://arxiv.org/abs/2502.06858

[3] Gyawali, B. R., Achalla, S., Kallas, K., & Kumar, S. (2025). NaSh: Guardrails for an LLM-Powered Natural Language Shell. *arXiv:2506.13028*. https://arxiv.org/abs/2506.13028

[4] van Dam, H. G. W. (2025). A Multimodal GUI Architecture for Interfacing with LLM-Based Conversational Assistants. *arXiv:2510.06223*. https://arxiv.org/abs/2510.06223

[5] Padmanabha, A., Yuan, J., Gupta, J., Karachiwalla, Z., Majidi, C., Admoni, H., & Erickson, Z. (2024). VoicePilot: Harnessing LLMs as Speech Interfaces for Physically Assistive Robots. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology (UIST '24)*. ACM. https://doi.org/10.1145/3654777.3676401

[6] Mehendale, S., & Walishetti, A. (2024). DexAssist: A Voice-Enabled Dual-LLM Framework for Accessible Web Navigation. *arXiv:2411.12214*. https://arxiv.org/abs/2411.12214

[7] Deng, X., Gu, Y., Zheng, B., Chen, S., Stevens, S., Wang, B., Sun, H., & Su, Y. (2023). Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. https://arxiv.org/abs/2306.06070

[8] Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., Alon, U., & Neubig, G. (2024). WebArena: A Realistic Web Environment for Building Autonomous Agents. In *Proceedings of ICLR 2024*. https://arxiv.org/abs/2307.13854

[9] Zheng, B., Gou, B., Kil, J., Sun, H., & Su, Y. (2024). GPT-4V(ision) is a Generalist Web Agent, if Grounded. In *Proceedings of ICML 2024*. https://arxiv.org/abs/2401.01614

[10] Rawles, C., Li, A., Rodriguez, D., Riva, O., & Lillicrap, T. (2023). Android in the Wild: A Large-Scale Dataset for Android Device Control. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. https://arxiv.org/abs/2307.10088

[11] Zhang, C., Yang, Z., Liu, J., Han, Y., Chen, X., Huang, Z., Fu, B., & Yu, G. (2023). AppAgent: Multimodal Agents as Smartphone Users. *arXiv:2312.13771*. https://arxiv.org/abs/2312.13771

[12] Wang, J., Xu, H., Ye, J., Yan, M., Shen, W., Zhang, J., Huang, F., & Sang, J. (2024). Mobile-Agent: Autonomous Multi-Modal Mobile Device Agent with Visual Perception. *arXiv:2401.16158*. https://arxiv.org/abs/2401.16158

[13] Hong, W., Wang, W., Lv, Q., Xu, J., Yu, W., Ji, J., Wang, Y., Wang, Z., Zhang, Y., Li, J., Xu, B., Dong, Y., Ding, M., & Tang, J. (2024). CogAgent: A Visual Language Model for GUI Agents. In *Proceedings of CVPR 2024*. https://arxiv.org/abs/2312.08914

[14] Niu, R., Li, J., Wang, S., Fu, Y., Hu, X., Leng, X., Kong, H., Chang, Y., & Wang, Q. (2024). ScreenAgent: A Vision Language Model-driven Computer Control Agent. *arXiv:2402.07945*. https://arxiv.org/abs/2402.07945

[15] Hui, T., Zhang, Y., Jiang, B., Sun, J., Cheng, F., Lin, X., & Yang, Y. (2025). WinClick: GUI Grounding with Multimodal Large Language Models. *arXiv:2503.04730*. https://arxiv.org/abs/2503.04730

[16] Zhang, C., He, S., Qian, J., Li, B., Li, L., Qin, S., Kang, Y., Ma, M., Liu, G., Lin, Q., Rajmohan, S., Zhang, D., & Zhang, Q. (2025). UFO²: The Desktop AgentOS. *arXiv:2504.14603*. https://arxiv.org/abs/2504.14603

[17] Xie, T., Zhang, D., Chen, J., Li, X., Zhao, S., Cao, R., Hua, T. J., Cheng, Z., Shin, D., Lei, F., Liu, Y., Xu, Y., Zhou, S., Savarese, S., Xiong, C., Zhong, V., & Yu, T. (2024). OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. https://arxiv.org/abs/2404.07972

[18] Zhang, C., He, S., Qian, J., Li, B., Li, L., Qin, S., Kang, Y., Ma, M., Liu, G., Lin, Q., Rajmohan, S., Zhang, D., & Zhang, Q. (2024). Large Language Model-Brained GUI Agents: A Survey. *arXiv:2411.18279*. https://arxiv.org/abs/2411.18279

[19] Li, Y., He, J., Zhou, X., Zhang, Y., & Baldridge, J. (2020). Mapping Natural Language Instructions to Mobile UI Action Sequences. In *Proceedings of ACL 2020*, 8198–8210. https://doi.org/10.18653/v1/2020.acl-main.729

[20] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of ICLR 2023*. https://arxiv.org/abs/2210.03629

[22] Han, W., Zeng, Z., Huang, J., Jiang, S., et al. (2025). GUIRoboTron-Speech: Towards Automated GUI Agents Based on Speech Instructions. *arXiv:2506.11127*. https://arxiv.org/abs/2506.11127

[23] Park, J., Tang, P., Das, S., Appalaraju, S., Singh, K. Y., Manmatha, R., & Ghadar, S. (2025). R-VLM: Region-aware Vision Language Model for Precise GUI Grounding. In *Findings of the Association for Computational Linguistics: ACL 2025*, 9669–9685. https://doi.org/10.18653/v1/2025.findings-acl.501

[24] Braun, V., & Clarke, V. (2006). Using thematic analysis in psychology. *Qualitative Research in Psychology, 3*(2), 77–101. https://doi.org/10.1191/1478088706qp063oa

[25] Jacobs, J., Lapon, J., & Naessens, V. (2026). Local LLMs for NL2Bash: A Large-Scale Open-Source Model Evaluation for Bash Command Generation. *NDSS Workshop on LLM Assisted Security and Trust Exploration (LAST-X 2026)*. https://www.ndss-symposium.org/wp-content/uploads/lastx2026-49.pdf

Appendices

This section contains full copies of the study instruments administered to participants, to be inserted once finalized during the pilot validation described in Section 2.3.

Appendix A: System Usability Scale (SUS)

[Placeholder — the finalized SUS instrument, as administered to participants after each condition, will be inserted here prior to submission.]

Appendix B: NASA Task Load Index (NASA-TLX)

[Placeholder — the finalized NASA-TLX instrument, as administered to participants after each condition, will be inserted here prior to submission.]